

Disaster Damage Assessment from Satellite Images

Evaluating U-Net, SAM2, and YOLO approaches using xBD/xView2 satellite imagery dataset

DEEP LEARNING

Jana Ehab | Mohamed Adel | Lana Galal | Farida Mohamed | Karen Hany

Made with GAMMA

The Problem: Slow and Risky Damage Assessment

Scale of Destruction

Natural disasters damage thousands of buildings in hours, overwhelming manual inspection teams.

Human Limitations

Ground surveys are slow, expensive, and often too dangerous to conduct immediately after a disaster.

The Opportunity

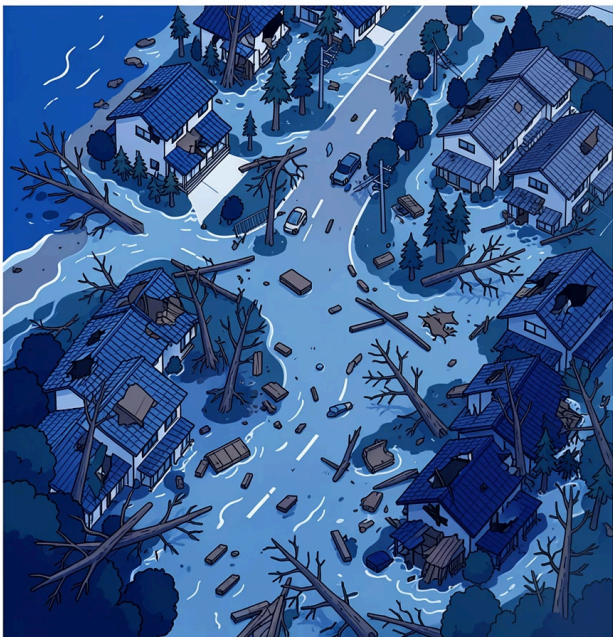
Satellite imagery provides wide-area, timely coverage – enabling automated, scalable damage assessment.



Dataset: xView2 / xBD



BEFORE



AFTER.

The **xView2/xBD dataset** is a benchmark for post-disaster building damage assessment, containing pre-disaster and post-disaster satellite image pairs with detailed annotations. Includes 19 disaster events.

Image Pairs

Pre- and post-disaster satellite images across multiple disaster types.

Pre/post-disaster
image pairs

9,168 pairs

Image size

1024 × 1024 RGB

Building annotations

850,736 buildings

4 Damage Classes

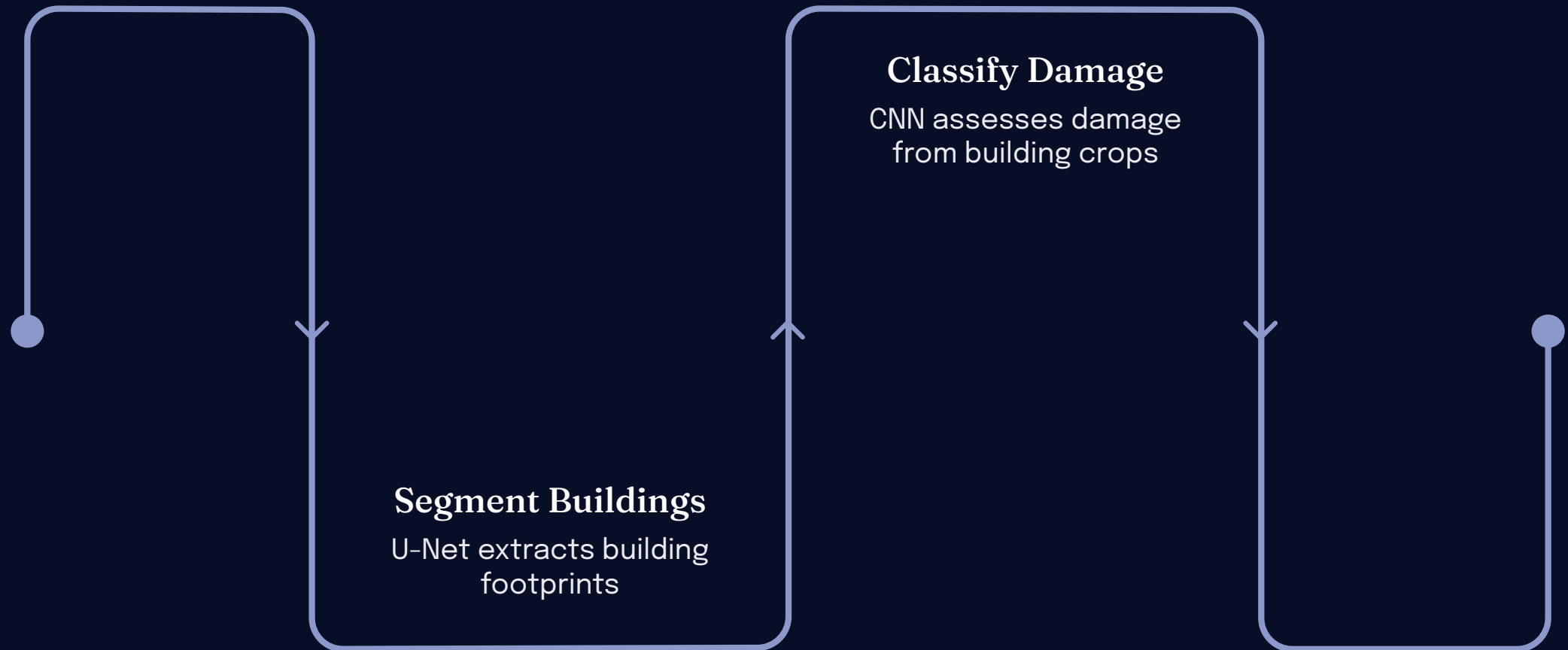
- No damage
- Minor damage
- Major damage
- Destroyed

Comparison of Three Deep Learning Models for Disaster Damage Assessment

Evaluating U-Net, SAM2, and YOLO approaches using xBD/xView2 satellite imagery dataset



Model 1: U-Net + CNN Pipeline



This classical two-stage approach first identifies building footprints using U-Net's encoder-decoder architecture, then extracts individual building regions for damage classification via a convolutional neural network.

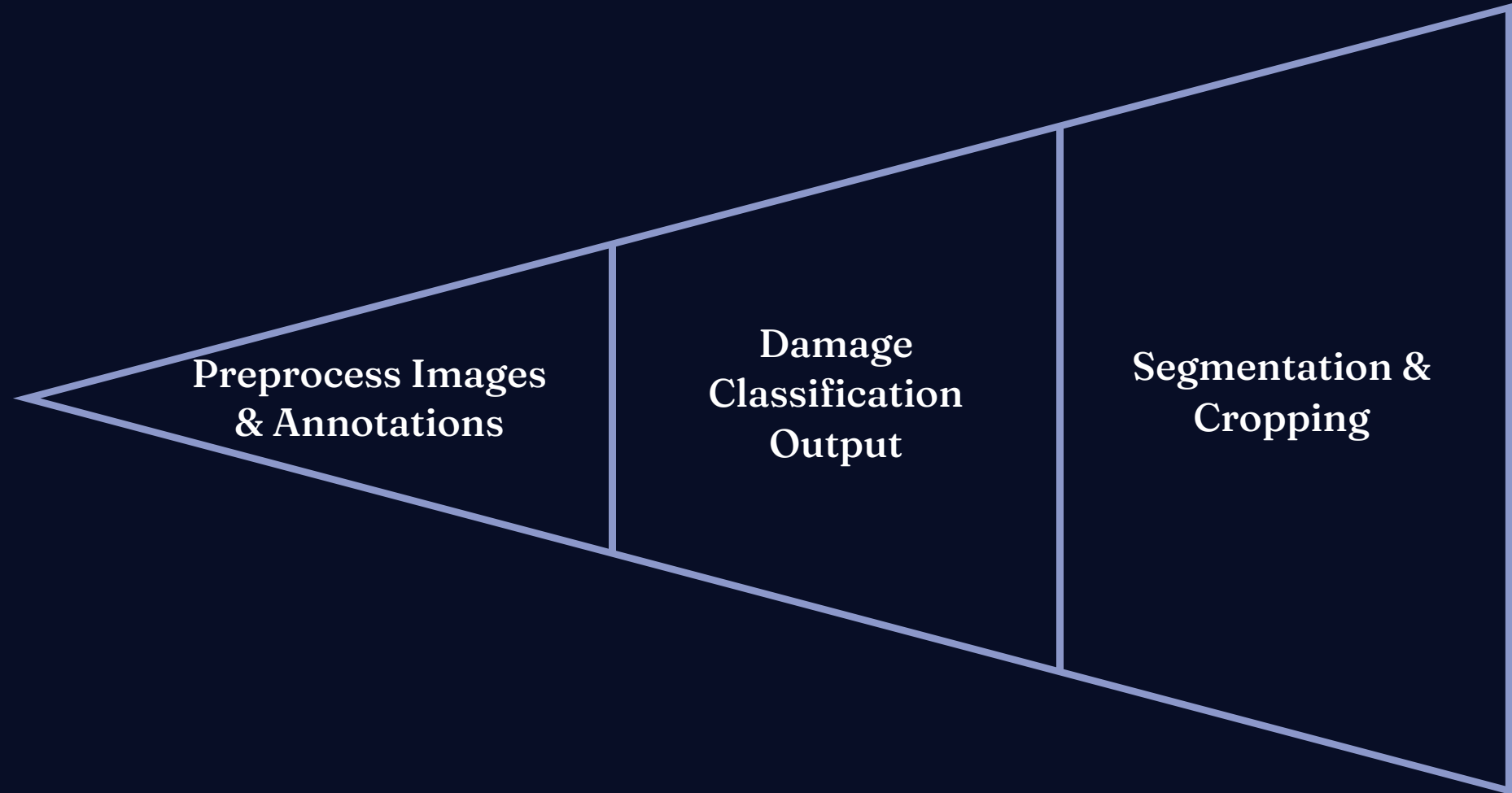
Strengths

Precise building boundary detection with pixel-level segmentation accuracy

Weaknesses

Two-stage pipeline increases computational overhead and inference time

End-to-End Project Pipeline



The pipeline processes raw satellite imagery through two deep learning stages – segmentation followed by classification – producing a per-building damage label for every structure in the scene.

Stage 1: U-Net Building Segmentation



Built from Scratch

The U-Net model is implemented manually – no pre-trained weights – giving full control over architecture and training.



Mask Generation

Building polygons from JSON annotations are rasterized into binary segmentation masks for supervised training.

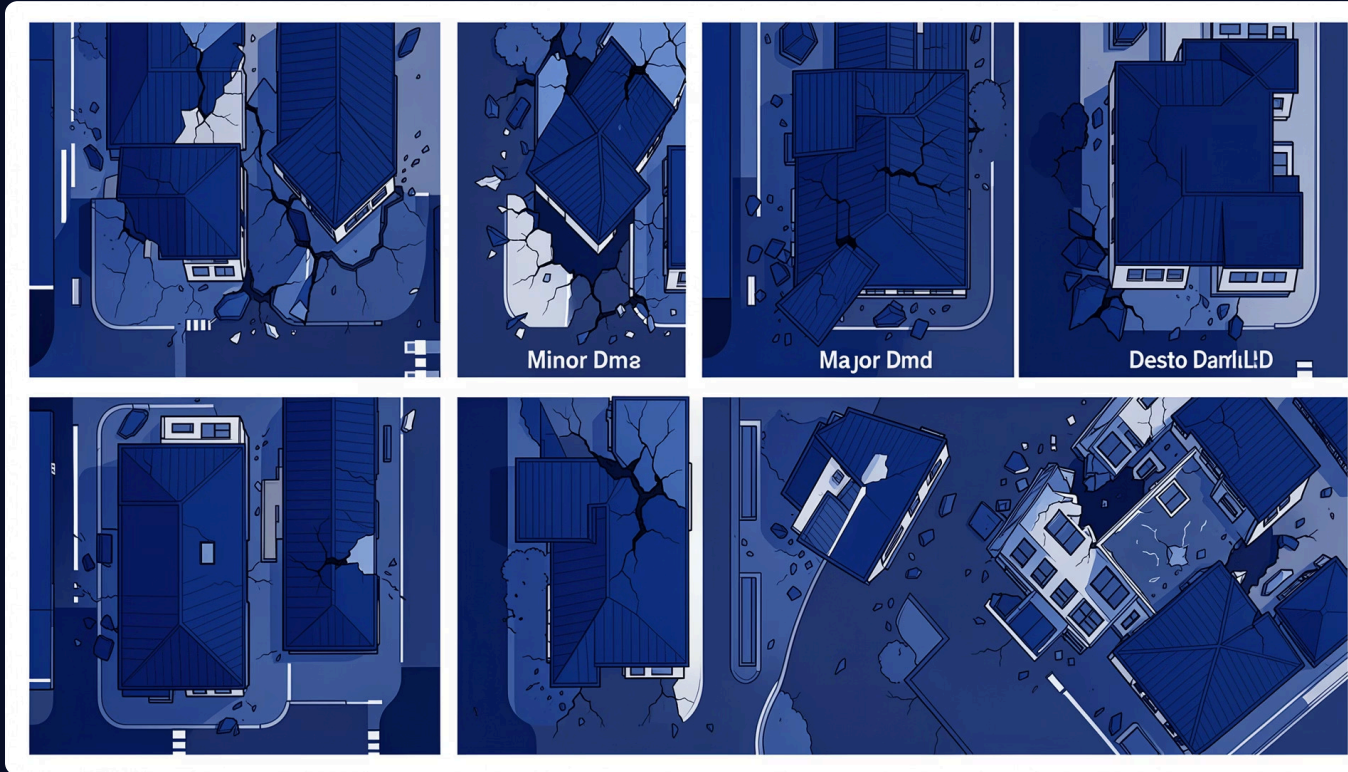


Preprocessing

Images are resized, augmented (flips, rotations), and normalized to improve generalization and reduce overfitting.



Stage 2: Damage Classification



After segmentation, **building-centered crops** are extracted from both pre- and post-disaster images. These paired crops are fed into a dual-input classifier that learns to compare before and after states.

Input

Pre-disaster crop + Post-disaster crop (same building)

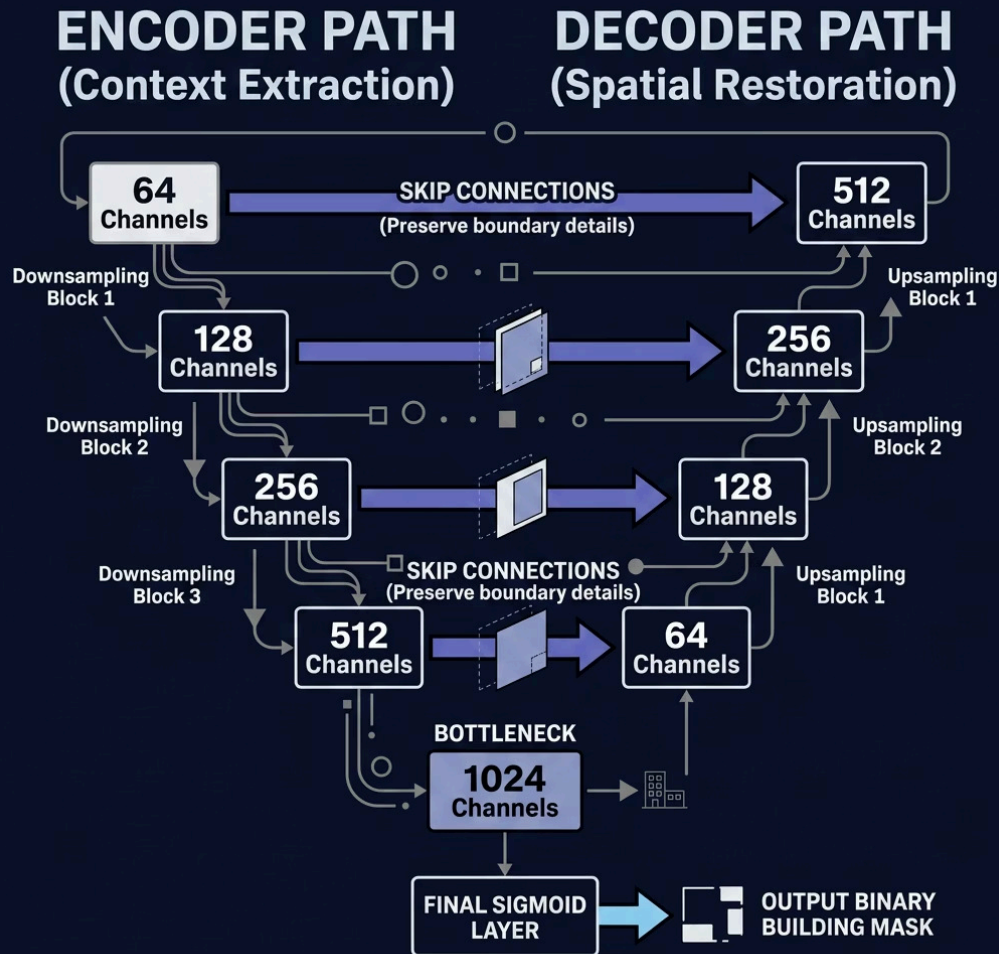
Fusion

Features concatenated + absolute difference computed

Output

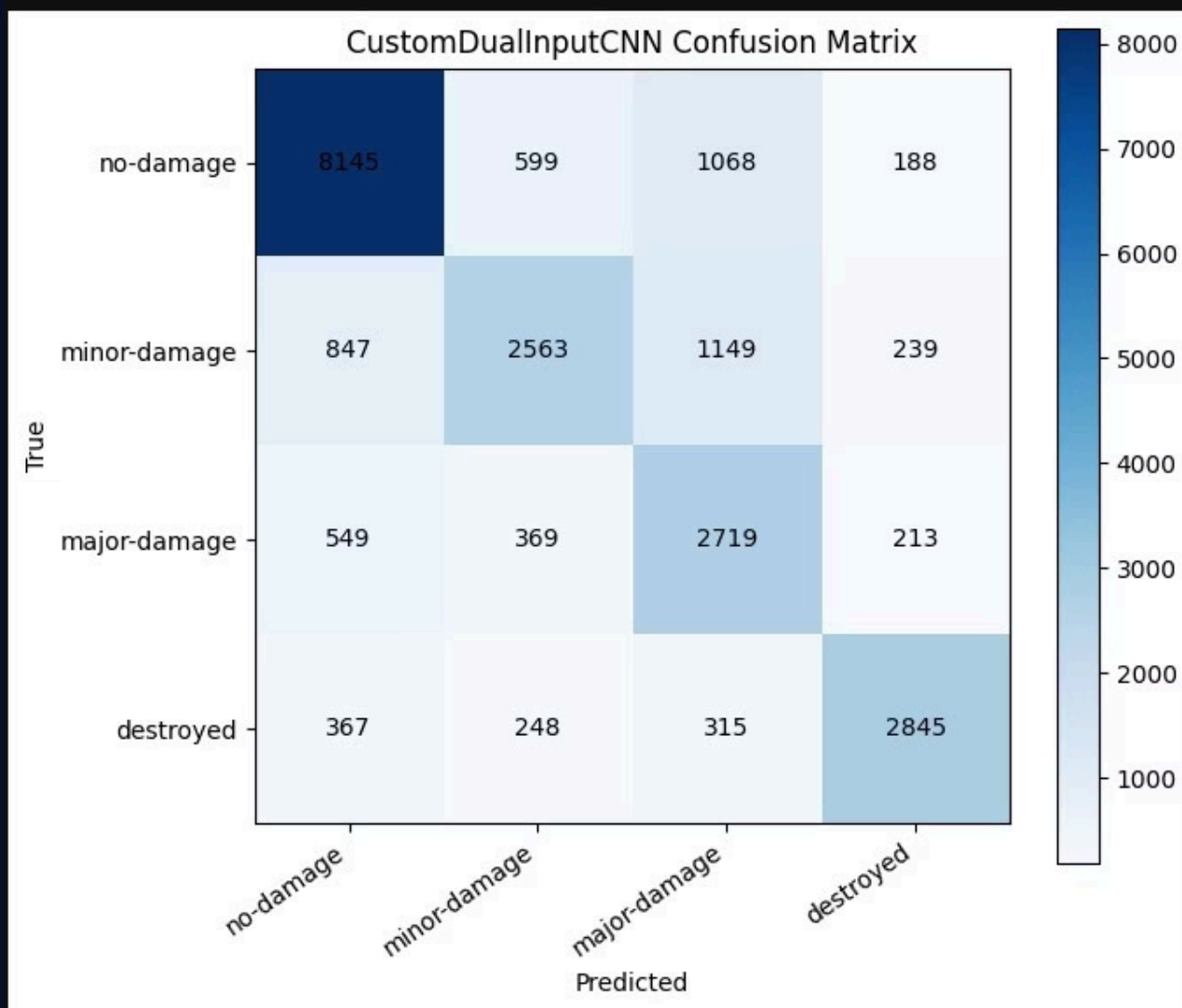
One of 4 damage classes per building

U-Net Architecture: Encoder–Decoder



How It Works

- **Encoder** downsamples the image, capturing broad spatial context at increasing abstraction levels.
- **Decoder** upsamples to restore full resolution, recovering precise building boundaries.
- **Skip connections** pass high-resolution features from encoder to decoder, preserving edge detail.



CustomDualInput Confusion Matrix

Model 2: SAM2 + Swin-Tiny Pipeline



This approach leverages Meta's Segment Anything Model 2 (SAM2) for robust building segmentation, followed by Swin Transformer-Tiny for sophisticated feature extraction and temporal comparison between pre-disaster and post-disaster building crops.

Strengths

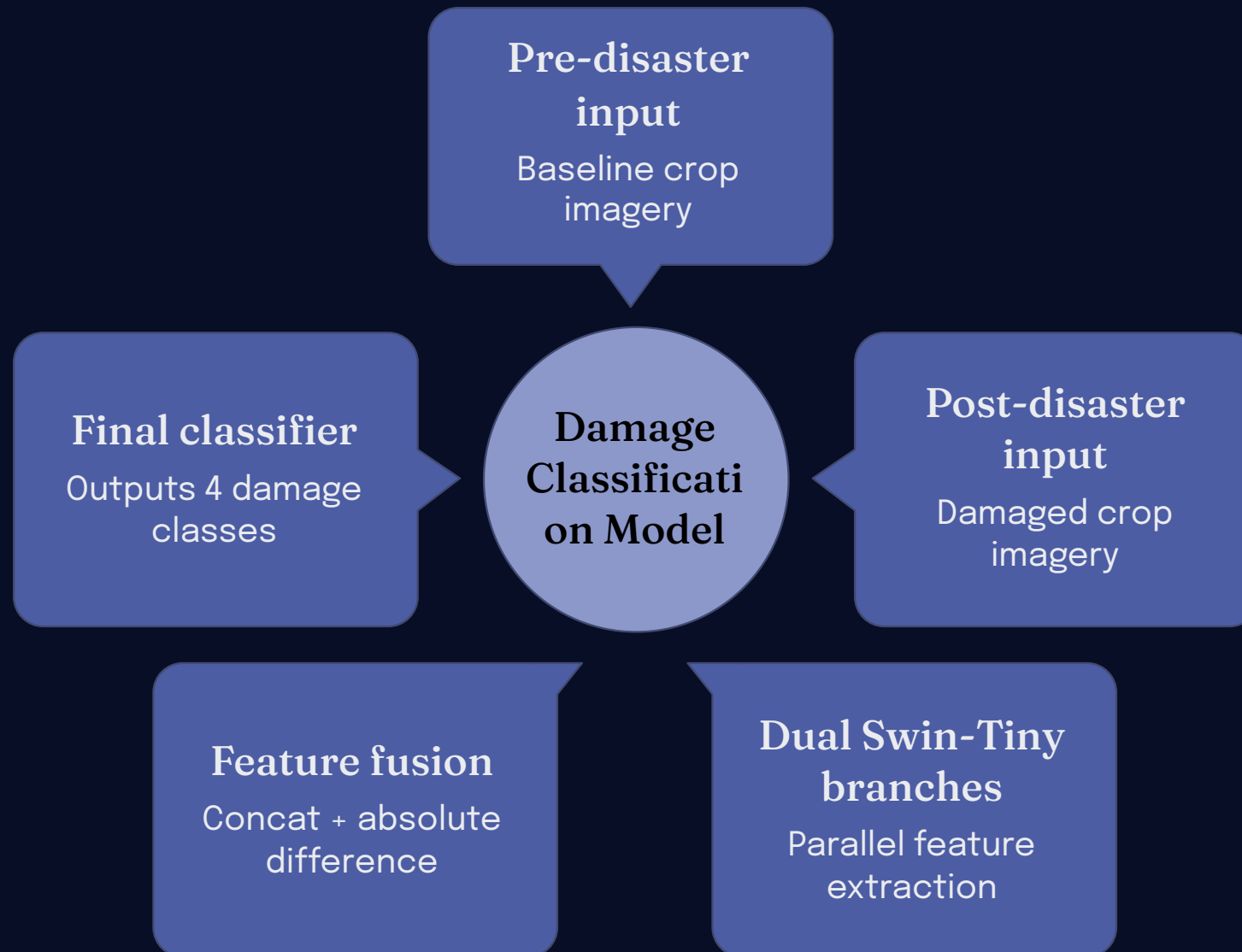
State-of-the-art segmentation quality combined with powerful transformer-based feature learning

Weaknesses

Requires prompt engineering or automated prompt generation for SAM2; higher computational complexity

Damage Classification Model

Dual-input **Swin Transformer-Tiny** architecture for comparative damage assessment.



Feature fusion combines pre-disaster features, post-disaster features, and their absolute difference – enabling the model to learn what changed.

Overall Workflow

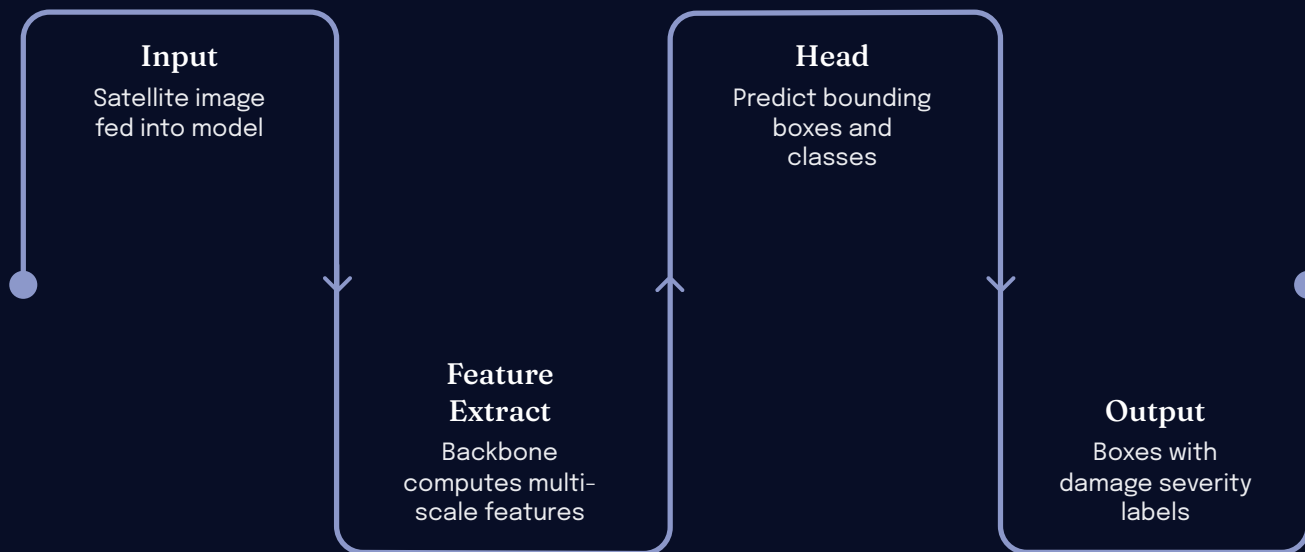
End-to-end pipeline from raw satellite imagery to damage class predictions.





Confusion Matrix & Segmentation

Model 3: YOLOv8-Based Damage Detection



YOLO's unified architecture performs simultaneous building localization and damage classification in a single inference step, outputting bounding boxes with associated damage severity labels.

Strengths

- Real-time inference for rapid disaster assessment
- Simultaneous localization and classification
- Efficient GPU-based deployment

Weaknesses

- Bounding boxes provide limited boundary precision
- Performance is affected by dense or overlapping buildings
- Less accurate than pixel-level segmentation masks

Why YOLO?

YOLOv8 provides a single-stage framework that simultaneously localizes buildings and predicts disaster damage severity in satellite imagery.



Real-Time Speed

Single-pass inference enables rapid disaster assessment for time-critical response workflows.



Joint Detection + Classification

Simultaneously detects building locations and predicts damage severity within a unified architecture.



Transfer Learning

Pretrained YOLO weights accelerate convergence and improve feature learning on xBD disaster data.



Built-In Localization

Integrated localization removes the need for a separate region proposal or external localization stage.

YOLO Data Preparation

1. Parse xBD Annotations

Read xBD JSON annotation files containing building polygons and disaster damage labels.

2. Extract Building Polygons

Extract building footprint coordinates and associated damage severity classes from polygon annotations.

3. Convert to YOLO Format

Convert polygon and bounding box coordinates into normalized YOLO label format with class IDs.

4. Generate Detection & Segmentation Labels

Create bounding box labels for object detection and polygon masks for segmentation training.

5. Resize and Organize Images

Standardize satellite image dimensions and organize images into train, validation, and test directories.

6. Dataset Balancing

Reduce overrepresented no-damage samples to mitigate class imbalance and improve model generalization.

7. YAML Configuration

Create YOLO YAML configuration files defining dataset paths, splits, and damage classes.

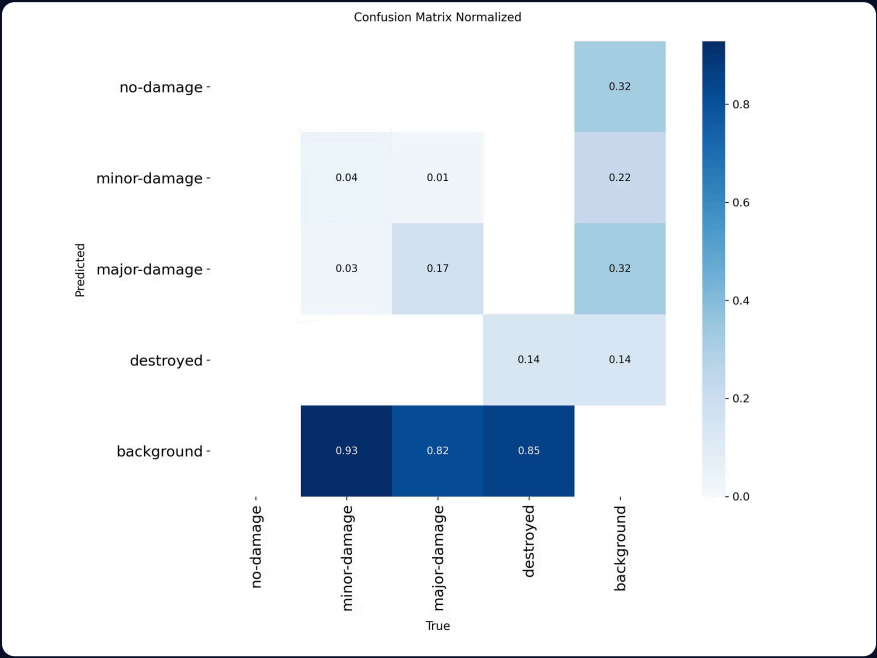
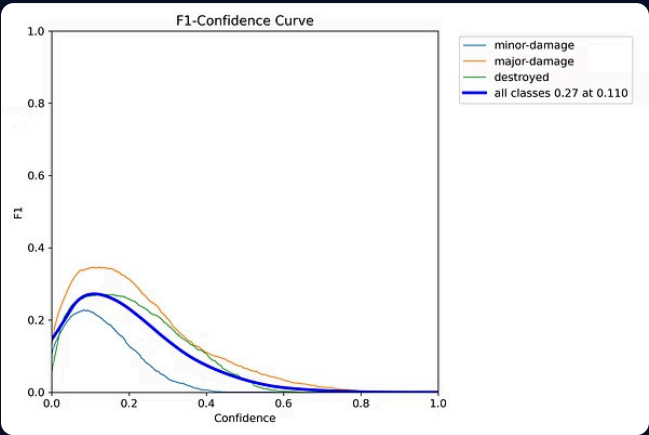
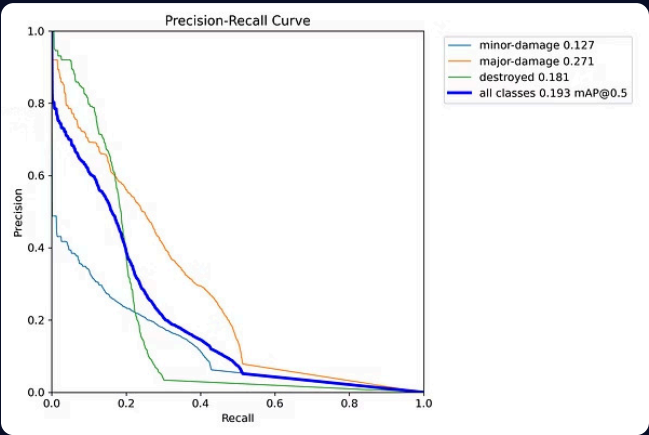
8. Final Dataset Split

Partition the processed dataset into training, validation, and testing subsets for evaluation.

YOLO Segmentation Metrics

The segmentation model achieved:

- 27% precision
- 21% recall
- 14.1% mask mAP@50



YOLO Object Detection Metrics

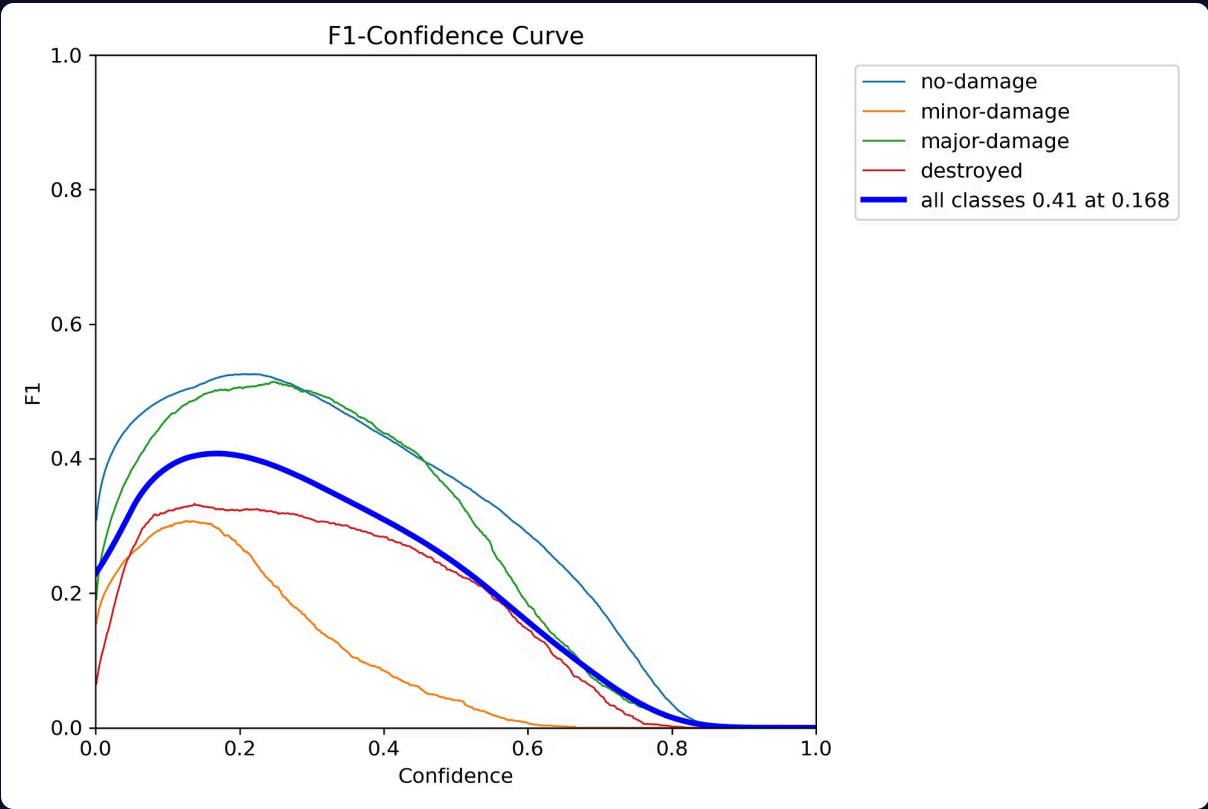
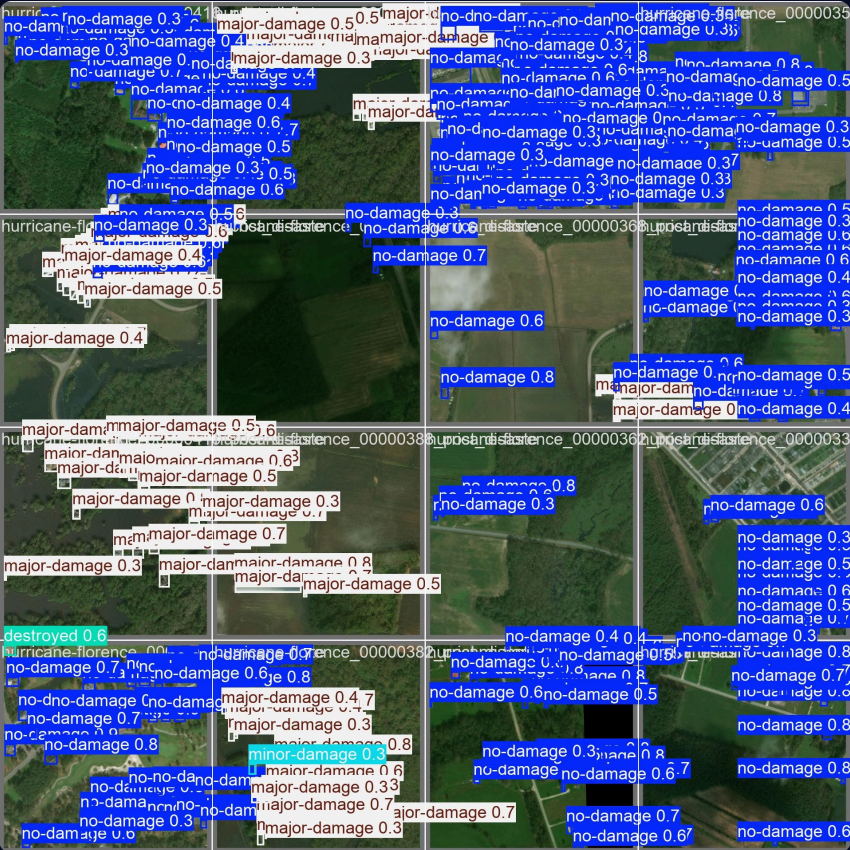
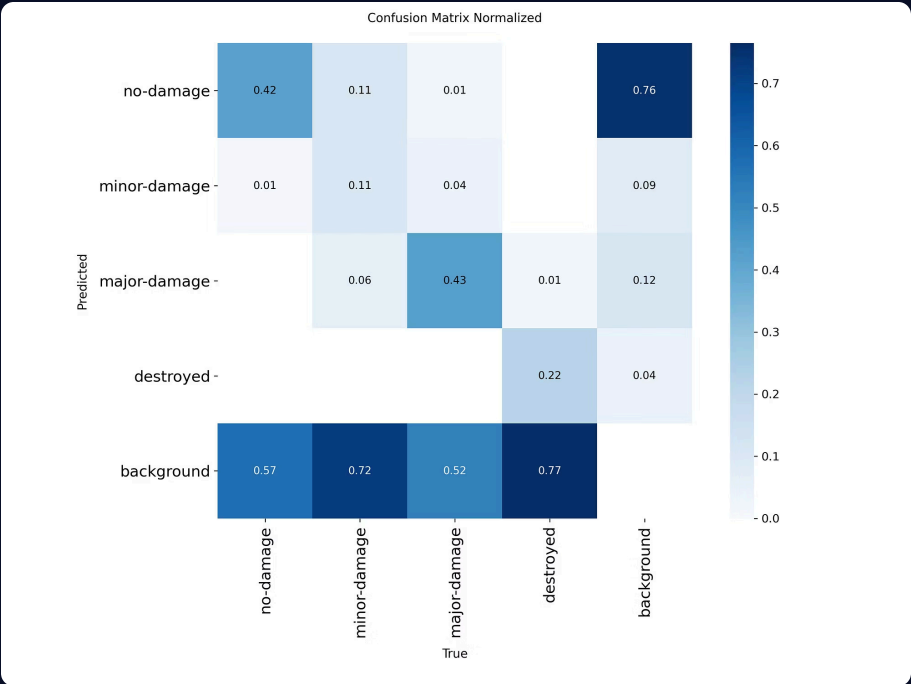
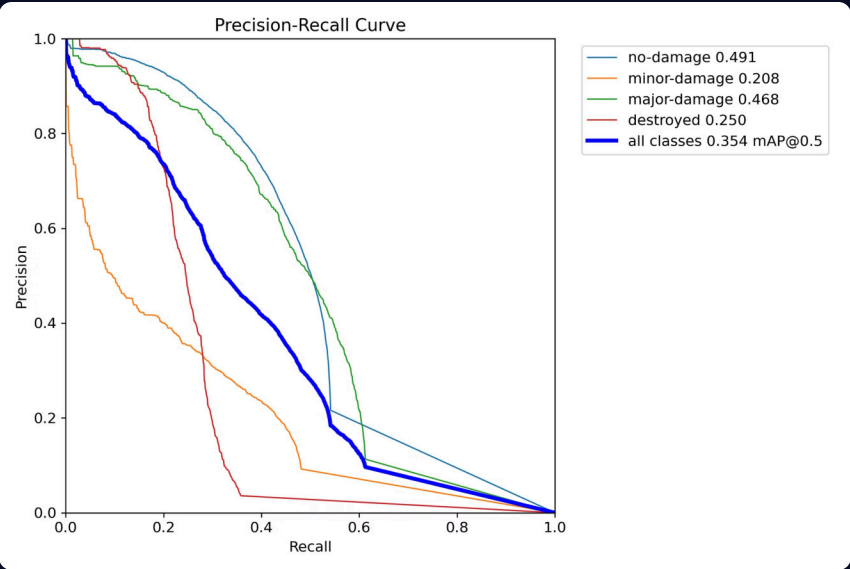
For object detection evaluation, the model achieved approximately:

Precision: 49.7%

Recall: 36.4%

mAP@50: 35.4%

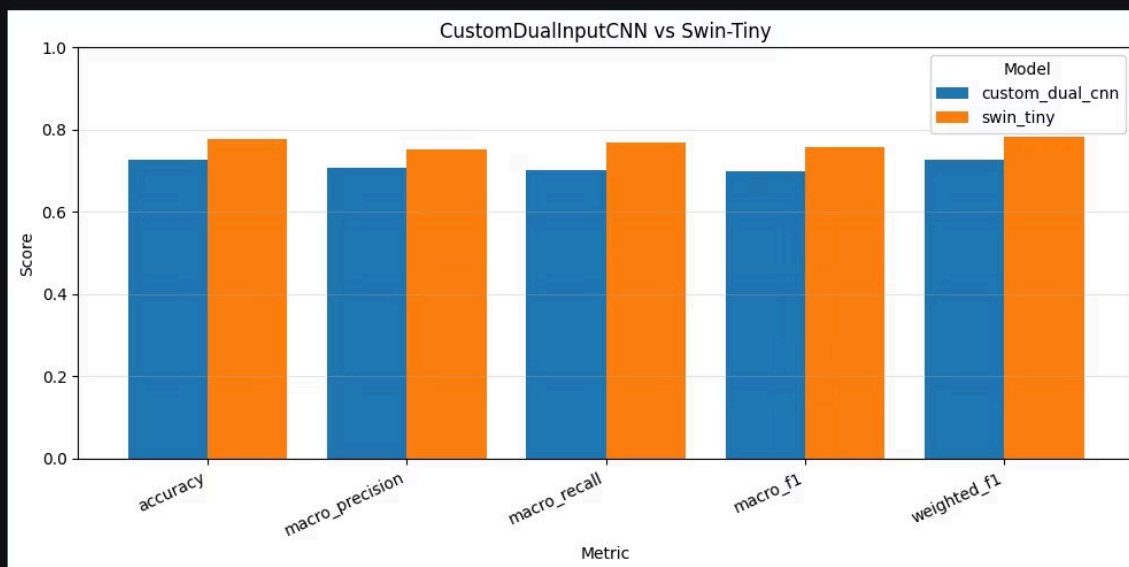
mAP@50-95: 16.6%



	model_name	accuracy	macro_precision	macro_recall	macro_f1	weighted_f1	num_samples	checkpoint_path	pickle_path
0	custom_dual_cnn	0.725683	0.708612	0.702139	0.699293	0.727343	22423.0	D:\Deeplearning project\runs\checkpoints\best_...	NaN
1	swin_tiny	0.777550	0.753391	0.768269	0.757549	0.781889	NaN	D:\Deeplearning project\runs\checkpoints\best_...	D:\Deeplearning project\runs\pickles\best_swin...

CustomDualInputCNN minus Swin-Tiny:

	metric	custom_minus_swin_tiny
0	accuracy	-0.051866
1	macro_precision	-0.044779
2	macro_recall	-0.06613
3	macro_f1	-0.058256
4	weighted_f1	-0.054546



Swim-Tiny VS Custom CNN

Related Papers & Projects

[On the Deployment of Post-Disaster Building Damage Assessment Tools using Satellite Imagery: A Deep Learning Approach](#) | [Request PDF](#)

Model / Method	Building Localization / Segmentation Accuracy	Damage Classification Accuracy
Baseline model from Gupta et al. [26]	0.79 BLD	0.03 DMG
Proposed Siamese U-Net CNN, compared against Gupta et al. [26]	0.74 BLD	0.58 DMG

Project Limitations Overview

Despite strong potential, several limitations are inherent to disaster-related AI projects.

Limited Dataset Diversity

xBD/xView2 may not represent all disaster types, regions, or building styles

Class Imbalance

Damage categories are unevenly distributed, biasing model predictions

Minor Damage Detection

Subtle damage is visually ambiguous and difficult to distinguish from no-damage

Image Quality Dependence

Clouds, shadows, blur, and low resolution degrade model performance

High Computational Cost

Some models require significant GPU resources and training time

Limited Deployment Testing

Real-world testing on completely unseen disaster events remains limited

Model-Specific Weaknesses

Each architecture carries distinct trade-offs in speed, accuracy, and deployment complexity.

Aspect	U-Net + CNN	SAM2 + Swin-Tiny	YOLO
Speed	Slower two-stage pipeline	Higher compute cost	Fastest – real-time capable
Output	Precise segmentation masks	Advanced masks + features	Bounding boxes only
Weakness	Seg. errors affect classification	Needs prompt automation	Misses small/overlapping buildings
Deployment	Requires accurate masks	Complex pipeline	Less boundary detail

FUTURE WORK

Future Directions



Expand & Balance Data

Train on larger, more diverse datasets; improve class balance via augmentation and weighted loss



Broaden Architecture Comparison

Evaluate Faster R-CNN, Mask R-CNN, Vision Transformers, and ConvNeXt



Enhance Preprocessing

Add cloud removal, shadow correction, and image alignment to improve input quality



Deploy for Emergency Response

Deploy the best model in a web dashboard; test on completely unseen disaster events

CONCLUSION

Conclusion

U-Net + CNN

Best when precise segmentation masks are critical for downstream analysis

SAM2 + Swin-Tiny

Delivers advanced segmentation and strong feature comparison, but with higher complexity

YOLO

Fastest and most practical for real-time disaster response scenarios

Deep learning can support faster, safer, and more scalable disaster assessment – but more work is needed before real-world deployment.

This project demonstrates that AI can significantly support disaster response by turning satellite imagery into actionable damage information.